R1.04 — Architecture et Systèmes d'Exploitation : Gestion des Processus, Cycle de Vie et Signaux POSIX

Thème principal : Gestion des Processus, Multitâche Préemptif, Filiation PID/PPID (fork/exec) et Signaux POSIX

Synthèse exécutive : Ce cours fondamental de R1.04 détaille le fonctionnement de l'exécution des applications au sein des systèmes d'exploitation de type Unix/Linux. Il explicite le basculement d'un programme statique (fichier exécutable sur disque) vers un processus dynamique (instance active chargée en mémoire RAM). Pour orchestrer le fonctionnement simultané de multiples applications sur les cœurs du processeur, le noyau (kernel) s'appuie sur le multitâche préemptif et gère le cycle de vie des tâches selon un modèle à trois états (Élu, Prêt, Bloqué). Le cours décortique le bloc de contrôle de processus (PCB), l'arborescence généalogique parent-enfant (PID/PPID) issue du processus initial systemd/init (PID 1), le mécanisme biphase de création par fork() et exec(), ainsi que le système universel de communication inter-processus et de contrôle par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter une application informatique, le système d'exploitation doit faire basculer le code d'un état statique sur le disque dur vers un état actif en mémoire vive (RAM). Ce passage crée une entité dynamique appelée processus. Dans un système informatique moderne, le nombre de programmes à exécuter dépasse largement le nombre de cœurs processeurs physiques. Le noyau Linux orchestre la simultanéité des tâches grâce au multitâche préemptif : l'ordonnanceur (scheduler) du noyau ne laisse pas un processus occuper le processeur indéfiniment. Il découpe le temps processeur en tranches ultra-courtes (quanta de temps) qu'il alloue successivement aux processus. Le cycle de vie d'un processus s'articule autour de trois états majeurs : 1. Élu (Running) : Le processus occupe physiquement un cœur du processeur et ses instructions machine sont exécutées. 2. Prêt (Ready) : Le processus possède l'intégralité de sa structure en mémoire RAM et attend que l'ordonnanceur lui attribue son prochain quantum de temps processeur. 3. Bloqué (Waiting / Blocked) : Le processus déclenche une requête nécessitant une attente externe (par exemple une lecture de fichier sur le disque dur, la réception d'un paquet réseau ou l'attente d'une saisie utilisateur au clavier). Le noyau libère immédiatement le processeur pour un autre processus et place la tâche en attente d'événement. Dès que l'événement survient et que la donnée devient disponible, le processus ne retourne JAMAIS directement à l'état Élu : il bascule impérativement dans l'état Prêt pour être réinscrit dans la file d'attente de l'ordonnanceur.

- • Programme vs Processus : Un programme est un fichier binaire passif enregistré sur disque dur (ex: /usr/bin/ls). Un processus est l'instance active et dynamique de ce programme chargée en mémoire RAM, consommant des cycles CPU et disposant de ressources allouées.
- • Multitâche Préemptif & Ordonnanceur : Mécanisme par lequel le noyau (kernel) découpe le temps processeur en tranches ultra-courtes appelées quanta de temps (quelques millisecondes). L'ordonnanceur (scheduler) alloue et retire autoritairement la main du CPU aux processus pour simuler une exécution parallèle simultanée.
- • Process Control Block (PCB) : Structure de données interne au noyau regroupant toutes les métadonnées d'un processus : état du processeur, registres, carte mémoire (stack, heap, code, data), identifiants (PID, PPID, UID), priorité et descripteurs de fichiers ouverts.
- • Cycle de Vie à 3 États : 1. Élu (Running) : le processus occupe le CPU et exécute son code. 2. Prêt (Ready) : le processus possède l'intégralité de ses ressources RAM et attend que l'ordonnanceur lui attribue un quantum CPU. 3. Bloqué (Waiting/Blocked) : le processus est suspendu en attente d'une entrée/sortie (lecture disque, paquet réseau, frappe clavier). RÈGLE D'OR : Un processus Bloqué repasse TOUJOURS par l'état Prêt une fois l'événement survenu, il ne repasse JAMAIS directement à l'état Élu.

Cas concret enseignant : *L'analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparable à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en mémoire RAM sur le plan de travail, consommant du temps processeur CPU pour la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Filiation, Arborescence et Mécanismes de Création (fork et exec)

Au sein des systèmes d'exploitation Unix/Linux, aucun processus ne peut apparaître de manière spontanée. Tous les processus utilisateurs s'inscrivent dans une généalogie stricte. Lors de l'initialisation de l'ordinateur, le noyau démarre le tout premier processus utilisateur, nommé systemd (ou init sur d'anciennes distributions), qui reçoit de façon garantie le PID 1. Lorsqu'un utilisateur lance une commande dans un terminal Bash (comme 'ls' ou 'vim'), le shell interactif (processus parent) exécute la création de la sous-tâche en deux temps : 1. fork() : Le processus père exécute l'appel système fork(). Le noyau duplique alors le père à l'identique. Un nouvel identifiant (PID) est assigné au fils, et le PPID du fils prend la valeur du PID du père. 2. exec() : Immédiatement après la bifurcation, le processus fils appelle l'une des fonctions de la famille exec(). Le noyau remplace l'intégralité du code et des données hérités de l'enfant par les nouvelles instructions binaires du programme visé. Le processus conserve son PID mais exécute désormais le nouveau code.

- Arborescence et Identifiants (PID / PPID) : Chaque processus possède un Process Identifier (PID) unique attribué par le noyau et pointe vers le Parent Process ID (PPID) du processus qui l'a engendré. Tous les processus forment un arbre généalogique dont la racine absolue est le processus initial systemd ou init (PID 1).
- Appel système fork() : Fonction du noyau qui duplique le processus parent à l'identique pour créer un processus enfant possédant son propre PID et une copie exacte du contexte mémoire.
- Appel système exec() : Fonction du noyau qui remplace l'image mémoire du processus appelant par le code binaire d'un nouveau programme à exécuter.

3. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est adressé à un processus cible, le noyau interrompt immédiatement le fil séquentiel des instructions pour exécuter l'action requise. Face à un signal, trois comportements sont possibles pour le processus : 1. Exécuter l'action par défaut du système (souvent la terminaison de la tâche). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en associant une fonction personnalisée (le gestionnaire de signal). Cela permet par exemple à un traitement de texte d'intercepter SIGTERM pour afficher 'Voulez-vous enregistrer votre document ?' avant d'éteindre l'application. Toutefois, afin de préserver l'autorité absolue de l'administrateur système et permettre de neutraliser un programme défaillant ou bloqué dans une boucle infinie, la norme POSIX définit deux exceptions incapturables : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont traités en direct par le noyau lui-même. Aucun code applicatif ne peut les détourner ou les ignorer.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement système ou lui ordonner une instruction.
- Gestionnaire de signal (Signal Handler) : Routine logicielle enregistrée par le développeur pour intercepter (catch) un signal et en exécuter un traitement sur-mesure au lieu du comportement système par défaut.
- SIGINT (Signal 2) : Déclenché par la combinaison 'Ctrl + C' dans le terminal. Demande une interruption douce et modérée d'une tâche interactive.
- SIGTERM (Signal 15) : Signal d'arrêt propre envoyé par défaut par la commande 'kill '. Il invite le processus à libérer ses ressources, fermer ses connexions sockets et sauvegarder ses fichiers avant de s'éteindre.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel et immédiat exécuté directement par le noyau Linux. STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer par le programme.
- Job Control (SIGSTOP 19 / SIGCONT 18) : Raccourci 'Ctrl + Z' émettant SIGSTOP pour suspendre l'exécution d'une tâche ; les commandes 'fg' (foreground) ou 'bg' (background) émettent SIGCONT pour reprendre son exécution.

Cas concret enseignant : *Analogie de l'expulsion du bâtiment : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (Signal 9) représente l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Dans la syntaxe de la commande 'kill' sous Linux, le premier argument sans tiret est interprété comme un numéro de PID et non comme un signal. Ainsi, 'kill 9 1234' tente d'envoyer le signal par défaut (SIGTERM 15) simultanément au processus dont le PID est 9 ET au processus dont le PID est 1234. Comme le PID 9 n'existe généralement pas ou n'appartient pas à l'utilisateur courant, le shell retourne une erreur 'No such process'. Pour spécifier le signal SIGKILL (Signal 9), il faut impérativement précéder le numéro du signal d'un tiret : 'kill -9 1234' ou 'kill -SIGKILL 1234'.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' dans le système et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie (état 'Z' dans la commande ps) est un processus enfant qui a terminé son exécution mais dont le père n'a pas encore lu le code de retour de fin (via l'appel système wait()). Le processus Zombie ne consomme plus de mémoire RAM ni de ressources CPU, mais sa ligne subsiste dans la table des processus du noyau. Il est STRICTEMENT IMPOSSIBLE de détruire un zombie avec la commande kill (même avec kill -9) car le processus est déjà mort ! Pour éliminer un zombie, il faut soit envoyer un signal au processus PÈRE pour qu'il exécute wait() ou se termine, soit tuer le père : le processus zombie est alors adopté par le processus racine systemd/init (PID 1) qui nettoie immédiatement la table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' indique au shell Bash d'exécuter la tâche en arrière-plan (background). Le shell affiche instantanément l'identifiant de job (ex: [1]) ainsi que le Process ID attribué par le noyau (ex: PID 4521) et rend immédiatement l'invite de commande à l'utilisateur sans bloquer le terminal. Étape 2 : La commande 'ps -ef | grep sleep' filtre la liste globale de tous les processus exécutés sur le système. La sortie confirme que le processus 'sleep 600' est actif à l'état Ready/Waiting avec le PID 4521 et identifie son PPID (qui correspond au PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus 4521. L'utilitaire sleep ne possédant pas de gestionnaire spécifique pour capturer ou ignorer SIGTERM, l'action par défaut du système d'exploitation s'applique : le processus s'arrête proprement et libère ses ressources système.

```
#!/bin/bash
# Séquence de démonstration de la gestion des processus et signaux

# 1. Lancement de la commande sleep 600 en arrière-plan
sleep 600 &

# 2. Recherche et affichage du PID du processus sleep
ps -ef | grep "sleep"

# 3. Émission du signal SIGTERM (15) au processus identifié
# Remplacez <PID> par la valeur numérique obtenue (ex: 4521)
kill -15 4521

# 4. Vérification de la destruction propre du processus
ps -ef | grep "sleep"
```

Résultat : Le processus 'sleep 600' (PID 4521) est détruit de manière propre par le signal SIGTERM (Signal 15). Une vérification ultérieure au moyen de la commande 'ps -ef | grep sleep' confirme sa disparition complète de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le premier processus utilisateur est nommé 'systemd' (ou 'init' sur les anciennes distributions) et possède impérativement le PID 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère le signal SIGINT (Signal 2), qui demande l'interruption modérée du processus.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : La norme POSIX impose que SIGKILL soit directement et exclusivement pris en charge par le noyau (kernel) lui-même, afin d'offrir à l'administrateur un moyen infaillible d'arrêter n'importe quel processus bloqué ou récalcitrant.